



Web Technologies

WET2VO

Teil 2: PHP-Grundlagen Teil 1

Medientechnik und -design | SS 2026

Allgemeines zu PHP

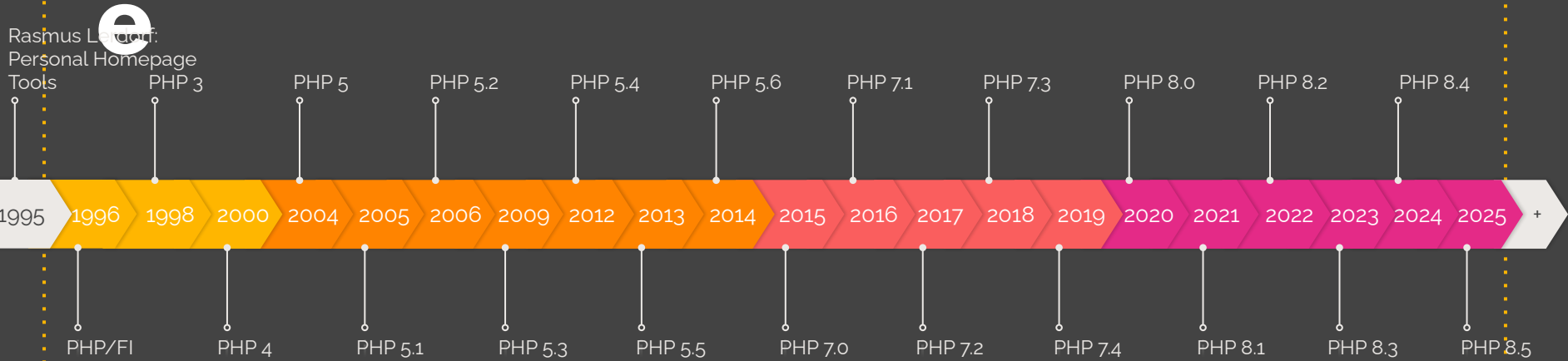
Grundsätzliches



php 8.5

PHP: **H**ypertext
Preprocessor

Timeline





Alternativen am Server

JavaScript

Node.js
Framework:
Express.js

Python

Frameworks:
Django, Flask

Java

Frameworks:
Spring, Struts

Scala

Framework:
Play
Framework

Ruby

Frameworks:
Ruby on Rails,
Sinatra

PHP im Jahr 2026?





71,7 Prozent

aller Seiten im Web verwenden PHP als serverseitige Technologie!

Quelle: w3techs.com (04/2026)

Serverseitige Webentwicklung

PHP im Big Picture

Warum serverseitig?



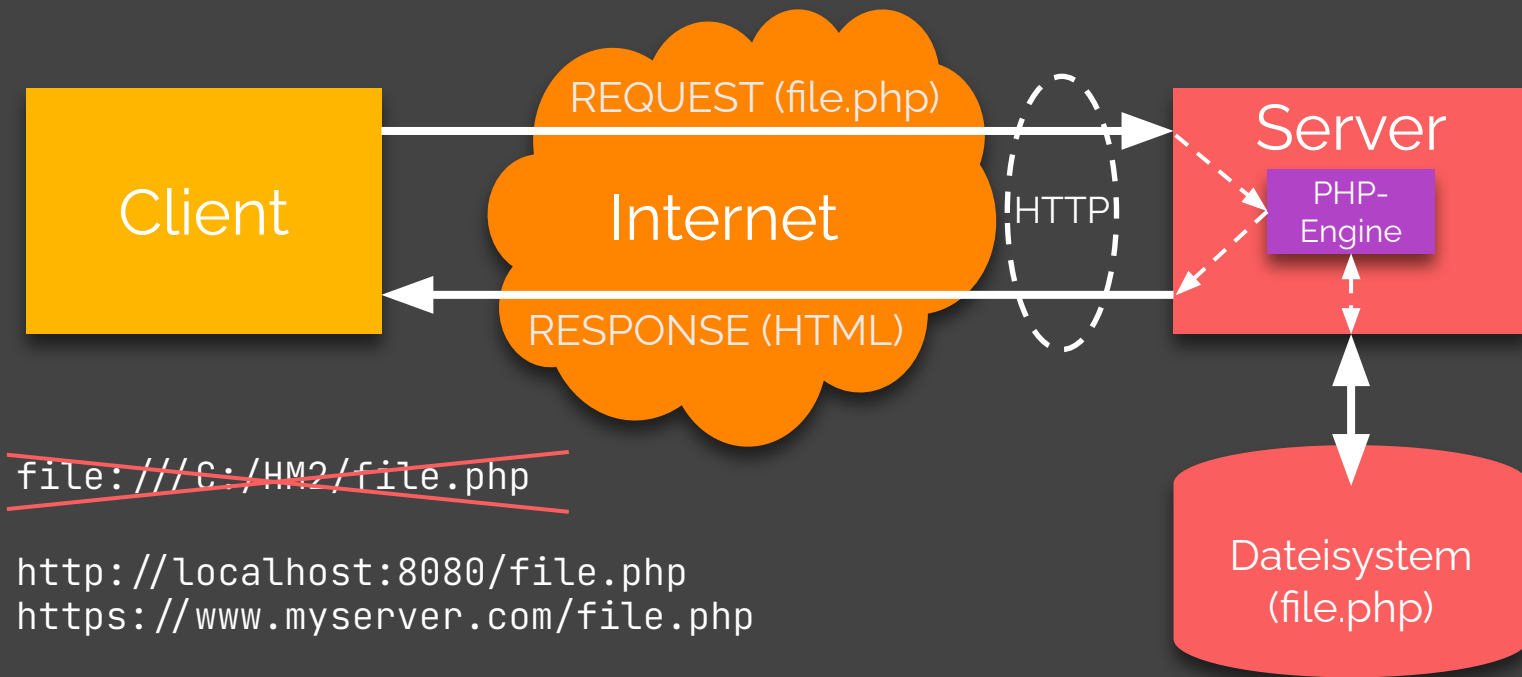
```
<!DOCTYPE html>
<html lang="de">
  <head>
    <meta charset="utf-8">
    <title>Titel der Seite</title>
  </head>
  <body>
    <p>Willkommen, Wolfgang!</p>
    <p>Heute ist der 15.04.2026.</p>
  </body>
</html>
```

Der Name wird z.B.
aus einem
abgeschickten
Formular oder einer
Datenbank abgefragt.

Das Datum soll immer
den aktuellen Tag
anzeigen.



Remember this?



Webserver mit Docker



fhooe-web-dock

compose.yaml + Dockerfiles

\$ docker compose up -d



Entwickler*in



docker

Docker Image(s)

Run

Run

Run



Docker Container



Docker Container



Docker Container

Webserver: <http://localhost:8080>

webapp → /

phpMyAdmin: <http://localhost:8082>

MariaDB: db / 3306 & localhost / 6033



Vom File zum Webdokument

1.

compose.yaml & Dockerfiles

Dateien liegen (gecloned aus
GitHub) z.B. in
C:\WET2\fhooo-web-dock

2.

Dateien im Shared Folder webapp

C:\WET2\fhooo-web-dock\webapp\file.php
(/var/www/html/file.php im Container)
C:\WET2\fhooo-web-dock\webapp\ue02\index.php
(/var/www/html/ue02/index.php im Container)

3.

Zugriff über Webserver

<http://localhost:8080/file.php>
<http://localhost:8080/ue01/index.php>

5

PHP Sprachgrundlagen

Let's talk!

Einbettung in HTML



```
<!DOCTYPE html>
<html lang="de">
  <head>
    <meta charset="utf-8">
    <title>Titel der Seite</title>
  </head>
  <body>
    <?php
    echo "<h1>Überschrift</h1>";
    echo "<p>Etwas Text</p>";
    ?>
  </body>
</html>
```

PHP Tags

```
<?php echo "PHP"; ?>
```

~~Short Open Tags~~

~~<? echo "PHP"; ?>~~

~~php.ini: short_open_tag=false~~

Short Echo Tags

```
<?="PHP" ?>
```

Unabhängig von short_open_tag=false



Ausgabe erzeugen

Sprachkonstrukt.

```
echo "Hallo Welt";  
echo "<h1>Überschrift</h1>";  
echo $text;
```

print()

Sprachkonstrukt, das sich wie eine Funktion verhält.

```
print("Hallo Welt");  
print("<h1>Überschrift</h1>");  
print($text);
```



Lexikalische Struktur

Groß- und Kleinschreibung:

Klassen, Funktionen, Sprachkonstrukte: `echo` = `ECHO` = `EcHo`

Variablennamen: `$name` vs. `$NAME` vs. `$NaMe`

Strichpunkt am Ende von Anweisungen:

```
$a = 3;
```

```
$b = 4;
```

```
// Kommentar
```

```
# Kommentar
```

```
/* Mehrzeiliger  
   Kommentar */
```


Datentypen

Zahlen, Zeichenketten und Co.

Zahlen



Ganzzahlen:

735 (dezimal)

0x2DF (hexadezimal)

0o1377 (oktal)

0b001011011111 (binär)

Gleitkommazahlen:

3.1415

-2345.67

6.02e23 ($6,02 \cdot 10^{23}$)

Seit PHP 7.4:

Numeric Literal Separator

1_000_000 (dezimal)

0xCAFE_F00D (hexadezimal)

0b0101_1111 (binär)

6.674_083e-11 (GKZ)



Boolean & null

Boolesche Werte

`true`, `false` (TRUE, FALSE)

Keinen Wert zuweisen

`null` (NULL)

Bei Typumwandlung auf boolean
(z.B. `if (0 == false) {...}`) immer false:

`0`, `0.0`, `""`, `"0"`, `[]`, `null`

Variablen

Werte speichern



Benennung

`$[a-zA-Z_\x7f-\xff]` `[a-zA-Z0-9_\x7f-\xff]`

Erstes Zeichen

Jedes weitere Zeichen

`\x7f-\xff`: Extended ASCII (127 bis 255)

`$counter`

`$nr_of_items`

`$MaxSpeed`

`$_temp`

Deklaration



Immer implizite Deklaration:

```
$counter;  
$nr_of_items;  
$MaxSpeed;
```

Implizite Deklaration und Zuweisung:

```
$number = 12;  
$name = "Igor";  
$sum = 3 + 5;
```

Loses

Typkonzept

```
$x = 42;
```

...

```
$x = "fourtytwo";
```



Schwache Typbindung



```
$a = 1;
if ($a == true) {
    echo "Typumwandlung in Boolean ist erfolgt!"; // Wird betreten
}
```

```
$b = "2";
$b += 4;
echo $b; // 6
```

```
$str = "14";
$i = (int)$str; // 14
```

```
$a = 1;
if ($a === true) {
    echo "Dieser Operator lässt sich nicht überlisten!"; // Nicht betreten
}
```


Zahlen/ Non-numeric Strings



Vergleich einer Zahl (z.B. 0) mit einem nicht-numerischen String (z.B. "foo") mithilfe von ==

Vor PHP 8: Umwandlung des **Strings in Zahl** (immer 0) und Vergleich.

Ab PHP 8: Umwandlung der **Zahl in String** und Vergleich.

```
if (0 == "foo") {  
    echo "true"; // Vor PHP 8  
} else {  
    echo "false"; // Ab PHP 8  
}
```

Operatoren

Mit Werten arbeiten

Zuweisung



Zuweisung:

```
$i = 0;
```

Zuweisung mit Operation:

```
$i += 5; (entspricht $i = $i + 5)
```

```
$i -= 3; (entspricht $i = $i - 3)
```

```
$i *= 6; (entspricht $i = $i * 6)
```

```
$i /= 3.5; (entspricht $i = $i / 3.5)
```

```
$i %= 2; (entspricht $i = $i % 2)
```



Vergleich

Vergleich auf Wert:

```
3 != 4           // true
```

```
3 == 3          // true
```

Vergleich auf Wert und Typ:

```
3.0 !== 3       // true
```

```
3.0 === 3       // false
```

Vergleich auf Verhältnis:

```
3 < 4           // true
```

```
"hallo" >= "welt" // false
```



Mehr Vergleichsoperatoren

Conditional Operator

```
$x > 0 ? $x * $y : -$x * $y;
```

Null Coalescing Operator (nimm ersten Wert, wenn existiert, sonst zweiten):

```
$name = $vorname ?? "Nobody";
```

Null Coalescing Assignment Operator (weise zu, wenn nicht existiert):

```
$hero ??= "Superman";
```

```
$hero = $hero ?? "Superman";
```

```
if ($hero === null) {  
    $hero = "Superman";  
}
```

Spaceship Operator (vergleiche Werte):

```
echo 1 <=> 1; // 0
```

```
echo 1.5 <=> 2.5; // -1
```

```
echo "b" <=> "a"; // 1
```

Arithmetik



Arithmetische Operatoren:

1 + 2 // 3

1 - 2 // -1

3 * 4 // 12

5 / 2 // 2.5

5 % 2 // 1

3 ** 2 // 9

`$i++`, `++$i` // `$i + 1`

`$i--`, `--$i` // `$i - 1`

Logik



Logisches UND:

```
$a = (4 > 2) && (5 < 3); // false
```

Logisches ODER:

```
$c = ($a === false) || ($b === false); // true
```

Logisches NICHT:

```
$b = !$a; // false
```

Stringkonkatenation



```
$str = "Hallo " . "Welt";  
echo $str; // Hallo Welt
```

```
$str = "Hallo " + "Welt";  
echo $str; // Seit PHP 8: Fehler (vorher 0)
```

```
$first = 100;  
$second = 50;  
echo $first . $second; // 10050  
echo $first + $second; // 150
```



Kontrollstrukturen

Abläufe im Programm steuern



Verzweigungen: **if**

```
if ($name != "") {  
    $username = $name;  
} else {  
    $username = "JD";  
}
```



if-else/elseif

```
$n = 1;
```

```
if ($n === 1) {  
    echo "Die Zahl ist 1";  
} else  
    if ($n === 2) {  
        echo "Die Zahl ist 2";  
    } else {  
        echo "Weder 1 noch 2";  
    }  
}
```

```
if ($n === 1) {  
    echo "Die Zahl ist 1";  
} elseif ($n === 2) {  
    echo "Die Zahl ist 2";  
} else {  
    echo "Weder 1 noch 2";  
}
```

Verzweigungen: **switch**



```
switch($n) {  
    case 1:  
        echo "Die Zahl ist 1";  
        break;  
    case 2:  
        echo "Die Zahl ist 2";  
        break;  
    default:  
        echo "Die Zahl ist weder 1 noch 2";  
        break;  
}
```

Verzweigungen: **match**



Neu ab PHP 8: match

Ersetzt/vereinfacht switch

```
echo match ($n) {  
    1 => "Die Zahl ist 1",  
    2 => "Die Zahl ist 2",  
    default => "Die Zahl ist weder 1 noch 2"  
};
```



Schleifen

```
$count = 0;  
while ($count < 10) {  
    // Mach was hier ...  
    $count++;  
}
```

```
for ($count = 0;  
    $count < 10;  
    $count++) {  
    // Mach etwas hier ...  
}
```

Advanced Escaping



```
<?php
if ($wert === "Hallo") {
    echo "<p>Hallo :)</p>";
} else {
    echo "<p>Kein Hallo</p>";
}
?>
```

```
<?php
if ($wert === "Hallo") {
    ?>
    <p>Hallo :)</p>
    <?php
    } else {
        ?>
        <p>Kein Hallo</p>
        <?php
        }
        ?>
```

1

0

Strings

Zeichenketten



Single vs. double

Single-quoted

```
echo 'Hallo'; // Hallo
echo 'Hallo\nWelt'; // Hallo\nWelt
echo 'Wichtig: $text';
// Wichtig: $text
echo 'PHP ist \'toll\'';
// PHP ist 'toll'
```

Double-quoted

```
echo "Hallo"; // Hallo
echo "Hallo\nWelt"; // Hallo
// Welt
$text = "Variable interpoliert!";
echo "Wichtig: $text";
// Wichtig: Variable interpoliert!
echo "PHP ist \"toll\"";
// PHP ist "toll"
```

Double Quotes: **Curly**



```
$count = 3;  
echo "Dauer: {$count}min"; // Dauer: 3min
```

```
// Ohne curly brackets:  
echo "Dauer: $countmin"; // Dauer: 3min  
Warning: Undefined variable $countmin
```



Heredoc vs. Nowdoc

Heredoc

```
$name = "John Doe";  
$hdText = <<<EOT  
    <p class="bold">Dies ist  
    ein Text von $name, der  
    genau so durch die  
    Zeilenumbrüche im String  
    gespeichert wird.</p>  
EOT;
```

Nowdoc

```
$name = "John Doe";  
$ndText = <<<'EOT'  
    <p class="bold">Dies ist  
    ein Text von $name,  
    der genau so durch die  
    Zeilenumbrüche im String  
    gespeichert wird.</p>  
EOT;
```

Stringfunktionen



Klassische Stringfunktionen vs. **Multibyte**-Stringfunktionen

Multibyte, wann immer möglich!

Beispiel: Länge eines Strings

```
$str = "Hallo";  
echo strlen($str); // 5
```

```
$str = "Hällo";  
echo strlen($str); // 6  
echo mb_strlen($str); // 5
```

Stringfunktionen Teil 1



```
$str = "Hallo Welt";  
echo $str[0];           // H  
echo $str[3];           // l  
  
echo mb_substr($str, 2, 3); // llo  
echo mb_substr($str, -2);  // lt  
echo mb_substr($str, -3, 2); // el  
echo mb_substr($str, 2, -3); // llo W  
  
$str = trim(" Hallo Welt "); // Hallo Welt  
$more = trim("!-Hallo Welta", "a-!"); // Hallo Welt
```

Stringfunktionen Teil 2



```
$value1 = 3;  
$value2 = "3";  
$name1 = "Fred";  
$name2 = "Wilma";
```

Vergleich mit strcmp (Umwand. In String)

```
echo strcmp($value1, $value2); // 0
```

Vergleich auf Wert (==, Umwand. in Int.)

```
if ($value1 == $value2) {  
    echo "Wert gleich";  
}
```

Vergleich auf Wert und Typ (===)

```
if ($value1 === $value2) {  
    echo "Wert und Typ gleich"; // Nope  
}
```

Vergleich zweier Strings

```
if ($name1 < $name2) {  
    echo "Fred kommt vor Wilma";  
}
```

Vergleich auf Wert (<, Umwand. in Int./String)

```
if ($name1 < $value1) {  
    echo "Fred < 3"; // Vor PHP 8 falsch!  
}
```

Vergleich mit strcmp (verlässlicher)

```
echo strcmp($name1, $value1); // 19
```

Stringfunktionen Teil 3



```
$text = "testen von strings";  
$pos1 = mb_strpos($text, "st");           // 2  
$pos2 = mb_strpos($text, "st", 5);        // 11  
  
$str1 = mb_substr("Testen von Strings", "St"); // Strings  
$str2 = mb_substr("Testen von Strings", "St"); // sten von Strings
```

Stringfunktionen Teil 4



```
$text = "Hallo kleine Welt";
```

```
echo mb_convert_case($text, MB_CASE_UPPER); // HALLO KLEINE WELT
```

```
echo mb_convert_case($text, MB_CASE_LOWER); // hallo kleine welt
```

```
echo mb_convert_case($text, MB_CASE_TITLE); // Hallo Kleine Welt
```




Nächstes Mal in WET2VO

PHP-Grundlagen Teil 2 und Regular Expressions

Credits



- S. 3: PHP 8.5 Logo: <https://www.php.net/releases/8.5/en.php>
- S. 11: Docker Logo: <https://www.docker.com/company/newsroom/media-resources>